WEB TECHNOLOGY · 2304362L

Assignment 4

Server-Side Validation Application using Node.js

STUDENT DETAILS	ASSIGNMENT DETAILS
Name: Rohan Reshi **PRN:** 202401040034 **Batch:** WT1	**Practical:** No. 4 — Server-side Development **Technologies:** Node.js, HTML, CSS, JavaScript **Date:** 29 October 2026

Project overview

A responsive student registration website served by Node.js. The server validates submitted JSON and returns structured responses with appropriate HTTP status codes. The project demonstrates core HTTP handling, custom modules, file-system logging, events, and streams.

Prepared for academic submission

1. Aim and Problem Statement

The aim is to design, develop, and deploy a Node.js application that demonstrates server-side validation and core Node.js capabilities. The application accepts student registration details and checks them on the server before returning a success or validation-error response.

Problem statement

Client-side validation alone can be bypassed, so submitted information must also be checked by the server. This project demonstrates server-side checks, predictable HTTP responses, and clear validation feedback for a student registration form.

2. Objectives

• Create an HTTP server using Node.js core modules.
• Handle GET and POST requests.
• Validate fields on the server and return appropriate status codes.
• Demonstrate custom modules, file-system operations, events, and streams.
• Provide a responsive form interface and a README with setup instructions.

3. Technologies Used

Technology	Role in the project
Node.js http	Creates the server, routes requests, and returns HTTP status codes.
Custom modules	validator.js contains reusable validation rules; logger.js writes event records.
File System (fs)	Serves frontend assets and appends valid/invalid event records to a log file.
EventEmitter	Emits separate events for valid and invalid registration attempts.
Transform stream	Processes and trims the request body before JSON parsing.
HTML, CSS, JavaScript	Builds the responsive registration UI and sends POST requests to the Node server.

4. Design and Architecture

The browser serves the form from the Node.js HTTP server. On submission, the browser sends JSON to `POST /api/register`. The server parses the request, runs the custom validation module, emits an event, writes a log record, and returns JSON with a status code.

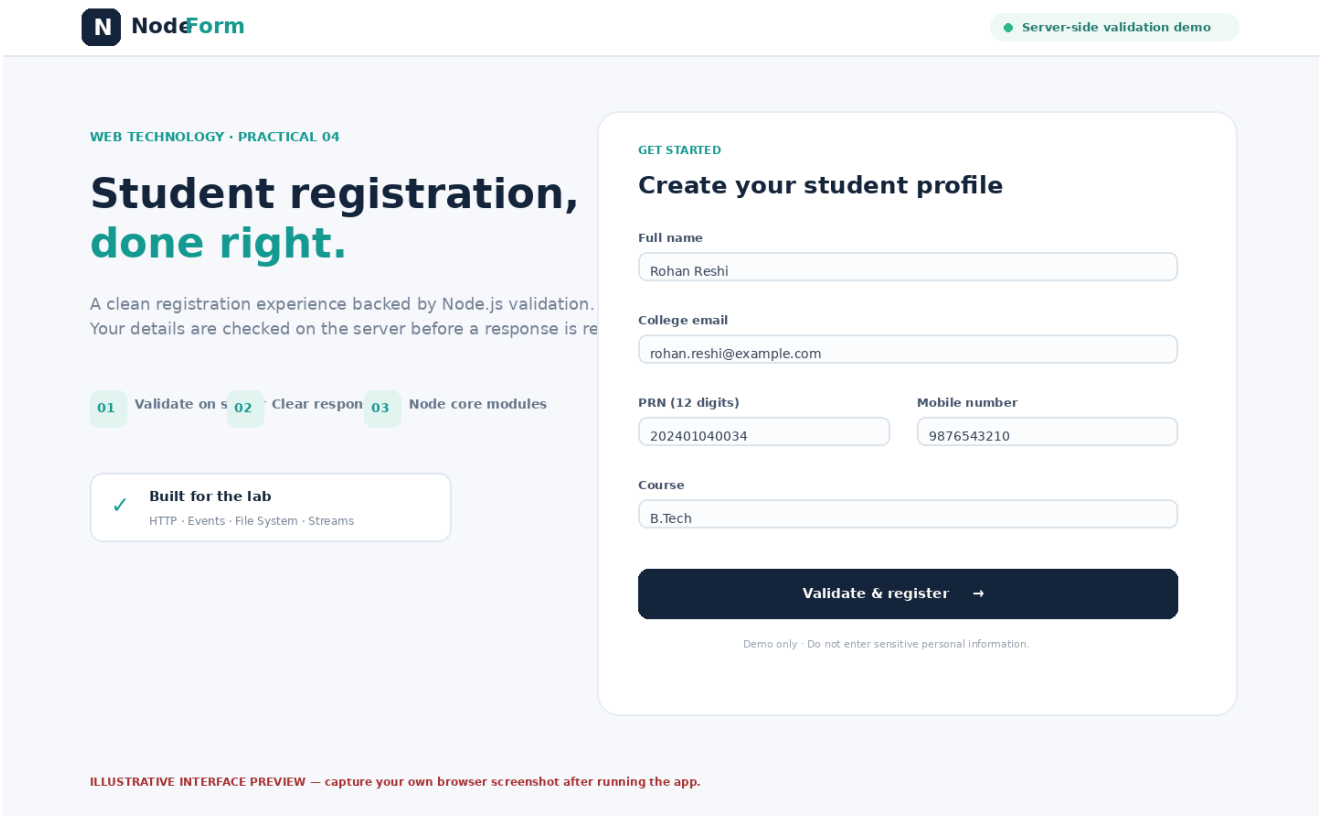
Browser form **HTML · CSS · JavaScript**
HTTP server **GET routes · POST route**
Request body → Transform stream → JSON parser
Custom validator **name · email · PRN · mobile · course**
Response JSON + HTTP status **201 valid · 422 invalid · 400 malformed JSON**
EventEmitter → File System log

5. Validation Rules

Field	Server-side rule
Name	At least 2 characters after trimming whitespace.
Email	Must match a standard email pattern; normalized to lowercase.
PRN	Exactly 12 digits.
Mobile	10 digits, beginning with 6, 7, 8, or 9.
Course	Must be one of B.Tech, M.Tech, BCA, or MCA.

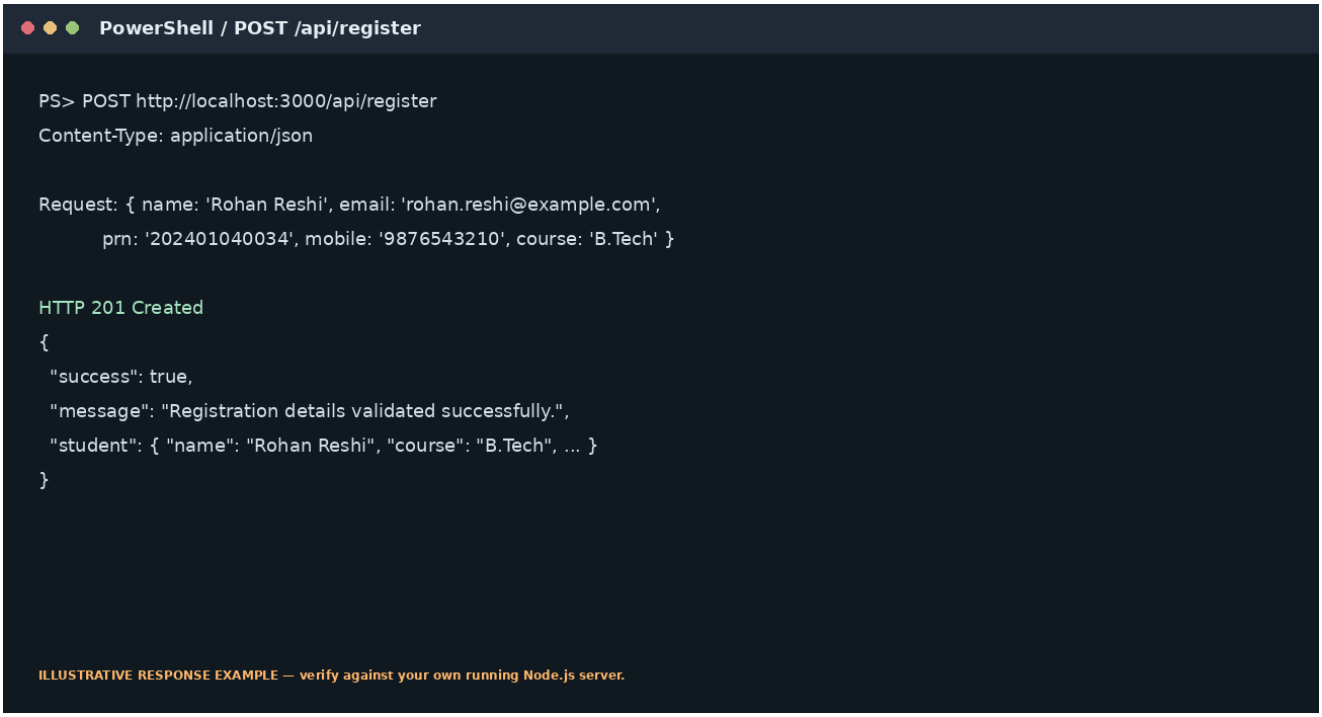
6. Output — Website Preview

Illustrative design preview of the student registration page. It shows the form layout and example values. This is a generated preview, not a browser capture from the student's computer.



7. Output — Valid Request Response

The local Node.js server was smoke-tested: the valid POST request returned HTTP 201 Created. The image below is a formatted illustrative response example rather than a direct terminal screenshot.



8. Output — Invalid Request Response

The local Node.js server was smoke-tested: invalid registration data returned HTTP 422 Unprocessable Entity with field-level error messages. The image is an illustrative response rendering, not a direct terminal capture.



9. Testing Results

Test	Expected result	Observed locally
GET /health	HTTP 200 and service status	Passed — 200
POST valid registration	HTTP 201 and success JSON	Passed — 201
POST invalid registration	HTTP 422 and validation errors	Passed — 422
JavaScript syntax check	No syntax errors	Passed for server and validator modules

10. How to Run and Deploy

Run locally: Extract the ZIP, open the folder in VS Code, open a terminal, run `npm start`, then visit `http://localhost:3000`. Use `GET /health` to verify the server.

Deployment: Push the source folder to a GitHub repository. Create a Node.js web service on Render or Railway and set the start command to `npm start`. The application reads the hosting provider's `PORT` environment variable. Add your actual GitHub and deployed URLs to Moodle only after testing them.

11. Limitations and Conclusion

The project demonstrates core Node.js HTTP handling, server-side validation, custom modules, file-system logging, events, and streams. The app is a learning demonstration; it does not persist registrations in a database, provide authentication, or send email. Logs may contain sample names and emails, so synthetic test values should be used. The project reinforced the importance of server-side checks and returning meaningful status codes.

12. AI Use Declaration

AI tool and prompt used: ChatGPT was used to help plan and draft a Node.js server-side validation example, UI, README, and report. The student should review and understand the submitted source before the viva.

One output checked and how: The HTTP endpoints were tested locally with a valid registration, an invalid registration, and a health-check request. The observed statuses were 201, 422, and 200 respectively.

One thing corrected or rejected, and why: Initial regular-expression escaping in the validator was corrected after the valid-request test returned a validation error. The tests were repeated after correction.

Student responsibility

The assignment instructions state that the student must be able to explain every part of the code during the demo/viva. Review the server routes, validation module, logger, event handling, stream processing, and deployment steps before submission.